PSYC 51.17: Models of Language and Communication

Lecture 25: Mixture of Experts & Efficiency

Scaling Efficiently with Sparse Models

PSYC 51.17: Models of Language and Communication

Week 9

Week 9

Today's Journey

What we'll cover

1. **The Scaling Problem**: Why bigger isn't always better
2. **Mixture of Experts**: Sparse activation for efficiency
3. **How MoE Works**: Routing, load balancing, training
4. **Real-World MoE**: Mixtral and production systems
5. **Other Efficiency Techniques**: Quantization, pruning, distillation

Week 9

The Scaling Dilemma

Larger models perform better... but at what cost?

Training a 175B parameter model (GPT-3 scale):

- Cost: $4-12 million in compute
- Energy: Equivalent to 120 homes for a year
- Time: Weeks to months on thousands of GPUs
- Inference: Slow and expensive ($0.002-0.02 per 1K tokens)
- Environmental: Massive carbon footprint

Can we get the benefits of scale without the full computational cost?

Dense vs Sparse Models

Dense Models (e.g., GPT-3):

```
Input Token
 ↓
[] ← All neurons active
[] ← 100% of parameters used
[] ← Every forward pass
 ↓
Output
```

- 175B parameters, 175B active
- High compute per token
- Simple architecture

Sparse Models (MoE):

```
Input Token
 ↓
[] ← Only 2 experts active
[] ← ~25% of parameters used
[] ← Per forward pass
 ↓
Output
```

- 56B total, 14B active (Mixtral)
- Low compute per token
- Complex routing logic

What is Mixture of Experts?

Mixture of Experts (MoE)

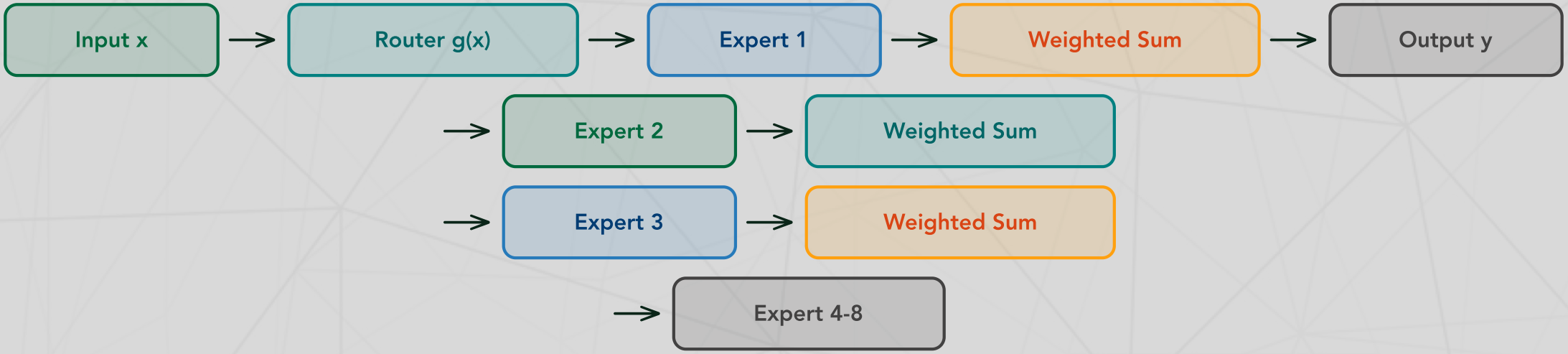
A neural network architecture where different "expert" sub-networks specialize in different parts of the input space, with a gating mechanism that routes inputs to the appropriate experts.

Key Components:

1. **Experts**: Multiple specialized feed-forward networks
2. **Router/Gate**: Learned function that assigns inputs to experts
3. **Sparse Activation**: Only top-k experts process each input

Intuition:

MoE Architecture



Concrete Example: Processing "def factorial(n):"

```
# Router computes scores for each expert
```

Typical setup: N=8-64 experts, but only top-2 are active per token

The Router Mechanism

Step-by-step routing for a single token:

```python
import torch.nn.functional as F

def route_token(x, router_weights, num_experts=8, k=2):
 """Route a token to top-k experts"""
 # Step 1: Compute routing scores (linear projection)
 # router_weights: (hidden_dim, num_experts)
 logits = x @ router_weights # Shape: (num_experts,)
 # Example: [-0.5, 2.1, 1.3, 0.2, -0.1, 0.0, -0.3, 0.1]

 # Step 2: Convert to probabilities
 probs = F.softmax(logits, dim=-1)
 # Example: [0.04, 0.52, 0.24, 0.08, 0.03, 0.03, 0.03, 0.03]

 # Step 3: Select top-k experts
 top_k_probs, top_k_indices = torch.topk(probs, k)
 # indices: [1, 2], probs: [0.52, 0.24]
```


continued...

The Router Mechanism

```
17
18  # Step 4: Normalize selected probabilities
19  top_k_probs = top_k_probs / top_k_probs.sum()
20  # [0.68, 0.32] # Now sums to 1
21
22  return top_k_indices, top_k_probs
23  # Expert 1 handles 68%, Expert 2 handles 32%
```


...continued

MoE in PyTorch (Simplified)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MoELayer(nn.Module):
 def __init__(self, d_model, num_experts, expert_capacity, k=2):
 super().__init__()
 self.num_experts = num_experts
 self.k = k # Top-k routing

 # Router
 self.gate = nn.Linear(d_model, num_experts)

 # Experts (simple FFN for each)
 self.experts = nn.ModuleList([
```


continued...

MoE in PyTorch (Simplified)

```
16  nn.Sequential(
17  nn.Linear(d_model, 4 * d_model),
18  nn.ReLU(),
19  nn.Linear(4 * d_model, d_model)
20  )
21  for _ in range(num_experts)
22  ])
23
24  def forward(self, x):
25  # x: (batch_size, seq_len, d_model)
26  batch_size, seq_len, d_model = x.shape
27
28  # Compute routing scores
29  router_logits = self.gate(x) # (batch, seq, num_experts)
30  router_probs = F.softmax(router_logits, dim=-1)
```


...continued...

MoE in PyTorch (Simplified)

```
31
32  # Select top-k experts
33  top_k_probs, top_k_indices = torch.topk(router_probs, self.k, dim=-1)
34  # top_k_probs: (batch, seq, k)
35  # top_k_indices: (batch, seq, k)
```


...continued

MoE Forward Pass (cont.)

```
# Initialize output
 output = torch.zeros_like(x)

 # Route to experts
 for i in range(self.k):
 # Get expert indices for this position
 expert_idx = top_k_indices[:, :, i] # (batch, seq)
 expert_weight = top_k_probs[:, :, i] # (batch, seq)

 # Process through each expert
 for expert_id in range(self.num_experts):
 # Mask for tokens routed to this expert
 mask = (expert_idx == expert_id)

 if mask.any():
 # Get tokens for this expert
 expert_input = x[mask]

```


continued...

MoE Forward Pass (cont.)

```
19  # Process through expert
20  expert_output = self.experts[expert_id](expert_input)
21
22  # Add weighted output
23  output[mask] += expert_weight[mask].unsqueeze(-1) * expert_output
24
25  return output
```

Note: This is simplified. Production implementations handle batching and load balancing more efficiently.

...continued

Problem: Without balancing, some experts get overused!

```
Expert Usage Distribution (Unbalanced):
```

Desired (Balanced):

```
F1: 12.5%
```

Why imbalance is bad:

- "Dead" experts waste parameters (never trained properly)
- Popular experts become bottlenecks during parallel inference
- Model loses expressiveness it paid for in memory

Load Balancing Solutions

Solution 1: Auxiliary Loss (Penalize Imbalance)

```
def compute_load_balance_loss(router_probs, expert_assignments,
alpha=0.01):
 """Add penalty to main loss for imbalanced routing"""
 num_experts = router_probs.shape[-1]

```

Solution 2: Expert Capacity Limits

```
capacity = (batch_size * seq_len) // num_experts * capacity_factor #
e.g., 1.25x
```

2. **Expert Imbalance**

- Some experts undertrained
- *Solution:* Balanced batching, capacity limits

3. **High Variance Gradients**

- Discrete routing decisions
- *Solution:* Softmax gating, larger batch sizes

Memory and Communication

MoE Memory Requirements:

Challenge

All experts must be in memory, even if only 2 are active!

Example: Mixtral 8x7B

- 8 experts × 7B parameters each = 56B total
- But only 2 experts active → 14B active parameters
- **Memory:** Need to store all 56B parameters
- **Compute:** Only process 14B parameters per token

Mixtral 8x7B

Mixtral (Jiang et al., 2024)

A state-of-the-art sparse MoE model from Mistral AI with 8 experts, each 7B parameters.

Architecture:

- **Total parameters**: 47B (8 experts × 7B, minus shared layers)
- **Active parameters**: 13B (only 2 experts active per token)
- **Layers**: 32 transformer blocks
- **Context**: 32K tokens
- **Vocabulary**: 32K tokens (SentencePiece)

Mixtral Performance

Comparison with dense models:

Model	Total Params	Active Params	MMLU Score	Speed vs 70B
Llama 2 13B	13B	13B	55.0	5× faster
Mixtral 8x7B	**47B**	**13B**	**70.6**	**5× faster**
Llama 2 70B	70B	70B	69.7	1× (baseline)
GPT-3.5	~175B	~175B	70.0	N/A (API)

The Magic of MoE:

```
1 Mixtral achieves:
```

Catch

Still need memory for all 47B params. Savings are in

What Do Experts Learn?

Experts naturally specialize without explicit supervision!

Analyzed routing patterns in Mixtral:

```
Token Type → Most Active Experts
```

Concrete Example: Sentence routing

```
"The neural network learns via backpropagation"
```

Key Finding

Specialization is **emergent**. Nobody told Expert 2 to handle code!

Model Compression Methods

Beyond MoE: Making models smaller and faster

1. Quantization

```
# Original: 32-bit float (4 bytes/param)
weight = 0.123456789 # Full precision

# INT8: 8-bit integer (1 byte/param)
weight_int8 = 31 # Scaled + quantized
# 4× memory reduction!
```

2. Knowledge Distillation

```
# Teacher: Large model (GPT-4)
# Student: Small model (GPT-2)

teacher_output = gpt4(input)
student_output = gpt2(input)

# Train student to match
```

Inference Optimizations

Making generation faster:

1. KV Cache (Essential)

```
# Without cache: Recompute all attention
# Token 100 attends to tokens 1-99
# = O(n) attention per token!

# With cache: Store previous K,V
cache = {}
for token in sequence:
```

3. Speculative Decoding

```
# Draft model (fast, small): 7B
draft_tokens = small_model.generate(5)
# ["The", "cat", "sat", "on", "mat"]

# Target model (slow, large): 70B
verified = large_model.verify(draft_tokens
# ["The", "cat", "sat", "on",
```

PSYC 51.17: Models of Language and Communication

State Space Models: Mamba

Alternative to Transformers with linear-time complexity

Transformer Attention: O(n)

```
Sequence length: 1K 4K 16K 64K
Compute (relative):
```

How SSMs Work (Simplified)

```
# State evolves with each token
state = initial_state
```

Efficiency Trade-off Landscape

Choosing the Right Technique for Your Use Case:

```
Quality
```

Technique	Best For	Trade-off
Dense Large	Maximum quality	Expensive, slow
MoE	Quality + speed	High memory
Quantization	Edge deployment	Slight quality loss
Distillation	Fixed tasks	Requires teacher
Pruning	Latency-critical	Irreversible

Decision Tree

Need max quality? → Dense. Need speed on GPU? → MoE. Need to run locally? → Quantized.

Environmental Impact

The carbon cost of large language models:

GPT-3	**502**	**112 cars for 1 year**
Llama 2	~539	120 cars for 1 year

Factors affecting carbon footprint:

- Model size (more parameters = more compute)

Democratizing Access

Should powerful AI be accessible to everyone, or only large organizations?

Current reality:

- Training GPT-3 scale model: $4-12M
- Requires 1000+ GPUs
- Only big tech companies can afford
- Creates AI "haves" and "have-nots"

Efficiency enables democratization:

Open vs Closed Models

Closed (GPT-4, Claude):

- Better safety control
- Monetization easier
- Protect IP
- Can update/improve
- No transparency
- Vendor lock-in
- Limited customization
- Privacy concerns

Open (Llama, Mixtral):

- Transparency
- Community innovation
- Full control
- No API costs
- Privacy (on-prem)
- Potential misuse
- Compute requirements
- Need ML expertise

2. **Conditional Computation**

- Adjust depth/width based on input difficulty
- Early exit for easy examples
- More compute for hard problems

3. **Neural Architecture Search**

- Automatically find efficient architectures

- Solutions: auxiliary losses, capacity limits

3. **Mixtral proves MoE works at scale**

- 70B-quality with 13B-cost
- Open weights accelerate adoption

4. **Many paths to efficiency**

1. **Shazeer et al. (2017)**: Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer
[arXiv]
2. **Jiang et al. (2024)**: Mixtral of Experts
[arXiv]

Recommended:

- Fedus et al. (2022): Switch Transformers [arXiv]
- Gu & Dao (2023): Mamba [arXiv]
- Dao et al. (2022): FlashAttention [arXiv]
- Strubell et al. (2019): Energy and Policy Considerations [arXiv]

Questions?

Next: Ethics, Bias, and Safety!

Week 9